Proyecto Fase 2: Biblioteca GG

Curso: Análisis de Sistemas II

Universidad Mariano Gálvez

Ingeniería en Sistemas - Sección A

Integrantes:

Gerardo Lucero -9989-23-21098

Giovanni Francia - 9989-23-15969

CONTEXTO DEL SISTEMA LEGADO

Descripción del sistema actual

- **Propósito:** Administración del catálogo de libros, usuarios, préstamos y devoluciones.
- **Procesos involucrados:** Autenticación, catálogo, préstamos, devoluciones.
- **Arquitectura actual:** Monolítica tradicional fuertemente acoplada.

Tecnologías actuales

Elemento	Tecnología Anterior
Lenguaje	PHP 5.6 / Java 7
Base de datos	PostgreSQL
Frontend	HTML, CSS, Vistas acopladas

Puntos de dolor identificados

- **Acoplamiento:** Lógica de negocio mezclada con la interfaz.
- **Cuellos de botella:** Búsquedas lentas por falta de índices.
- **Mantenibilidad:** Código espagueti que eleva el riesgo de bugs.
- **Seguridad:** Autenticación insegura basada en sesiones obsoletas.

ANÁLISIS DE ALTERNATIVAS ARQUITECTÓNICAS

Criterio	Monolito Tradicional	Microservicios	Monolito Modular (Elegido)
Escalabilidad	Baja	Alta	Media-Alta
Mantenibilidad	Baja	Media	Alta
Complejidad de Despliegue	Baja	Alta	Baja
Costos Operativos	Bajos	Altos	Bajos

Arquitectura Seleccionada: Monolito Modular con Node.js y Express.

Justificación: Separa estrictamente las responsabilidades por dominios (Rutas, Controladores y Modelos) sin la sobrecarga operativa y de infraestructura que exigen los microservicios, logrando un balance ideal para el MVP actual.

CONTRATO API – OPENAPI 3.0

Descripción de la API

- **Nombre:** BibliotecaGG RESTful API (v1.0.0)
- **Propósito:** Exponer servicios para clientes web interactuando de forma asíncrona.
- **Consumidores:** Aplicación cliente/Frontend.

Endpoints Principales (v1)

Método	Endpoint	Operación	Autenticación
POST	/api/v1/auth/login	Autenticación de usuario	No
GET	/api/v1/books	Listar/Buscar libros en catálogo	Opcional
POST	/api/v1/books	Registrar un nuevo libro	Sí (Bearer JWT)
POST	/api/v1/loans	Crear un nuevo préstamo	Sí (Bearer JWT)
PUT	/api/v1/loans/{id}/return	Registrar devolución	Sí (Bearer JWT)

Estructura de DTOs y Seguridad

- **Seguridad:** Se utiliza *Bearer Authentication* con tokens JWT (JSON Web Tokens). Las rutas protegidas validan el token para confirmar la identidad y rol del usuario.
- **Códigos de Respuesta (HTTP Status):**
  - 200 OK / 201 Created: Operaciones exitosas.
  - 400 Bad Request: Errores en payload (validación de datos).
  - 401 Unauthorized: Token inválido o ausente.
  - 500 Internal Server Error: Excepciones de servidor.

DISEÑO DE ARQUITECTURA C4

Nivel 1 - System Context: Los usuarios (Bibliotecarios y Lectores) interactúan con el Sistema Biblioteca GG a través de una interfaz cliente que se comunica con nuestra API central.

Nivel 2 y 3 - Contenedores y Componentes: La arquitectura monolítica modular en Node.js se descompone internamente en la carpeta src/ mediante:

- **Routes:** Enrutadores de Express que interceptan peticiones HTTP.
- **Controllers:** Manejan la lógica de negocio y respuesta al cliente.
- **Models (Sequelize):** Abstracción de datos que interactúa con PostgreSQL.

DISEÑO DE PERSISTENCIA DE DATOS

Tecnología Seleccionada

- **Motor:** PostgreSQL (Puerto 5432).
- **ORM:** Sequelize (versión 6.37).
- **Justificación:** PostgreSQL garantiza la integridad relacional y transacciones ACID necesarias para préstamos. Sequelize permite mapear la base de datos a objetos JavaScript, previniendo inyecciones SQL y agilizando el desarrollo.

Entidades Principales

Entidad	Responsabilidad	Atributos Principales
Users	Credenciales y roles.	id, email, password_hash, role
Books	Inventario de biblioteca.	id, isbn, title, author, available_copies
Loans	Histórico de préstamos.	id, user_id, book_id, issue_date, status

Gestión de la Base de Datos (Scripts)

A diferencia de sistemas legados basados puramente en scripts DDL (schema.sql), la creación de tablas se gestiona a nivel de código mediante la **sincronización de modelos de Sequelize** (sequelize.sync()) o mediante el uso de migraciones formales de ORM, lo que garantiza que la base de datos siempre esté sincronizada con el código fuente en Node.js.

ESTRATEGIA DE CACHÉ (Futuras Mejoras)

Aunque el MVP actual centraliza las operaciones en PostgreSQL a través de Sequelize, se propone como mejora arquitectónica futura la integración de **Redis** mediante un patrón *Cache-Aside*. Esto permitirá aliviar la carga de lecturas repetitivas (como el catálogo de libros), almacenando resultados en memoria con un TTL (Time to Live) definido, garantizando un escalamiento óptimo cuando incremente el volumen de usuarios.

MATRIZ DE TRAZABILIDAD

Necesidad / Requisito	Solución Implementada
Acoplamiento del sistema legado	Arquitectura basada en Rutas, Controladores y Modelos (Express.js)
Exposición de funcionalidades	Desarrollo de API RESTful documentada
Mecanismo de Autenticación	Implementación de JSON Web Tokens (JWT)
Gestión segura de credenciales	Uso de variables de entorno (.env) y hashing
Persistencia relacional	Integración con PostgreSQL vía Sequelize ORM

CONCLUSIONES

- **Superación del Monolito Legado:** Al reestructurar el sistema bajo el ecosistema de Node.js, Express y Sequelize, se ha logrado un código altamente mantenible y un despliegue ágil, eliminando la deuda técnica de la versión anterior.
- **Seguridad y Rendimiento:** La implementación de JWT previene los riesgos del manejo tradicional de sesiones, mientras que el ORM protege activamente contra vulnerabilidades de inyección de bases de datos.
- **Preparado para el Futuro:** El sistema actual (MVP) sienta bases firmes para que la adición de servicios como Redis o su eventual partición en microservicios reales sea una transición natural y no un rediseño completo.